

TD2 : Analyse de CVE

Exercice 1

A partir de la description suivante, répondre aux questions :

PHPMailer 6.1.8 through 6.4.0 allows object injection through Phar Deserialization via addAttachment with a UNC pathname. NOTE: this is similar to CVE-2018-19296, but arose because 6.1.8 fixed a functionality problem in which UNC pathnames were always considered unreadable by PHPMailer, even in safe contexts. As an unintended side effect, this fix eliminated the code that blocked addAttachment exploitation.

Q1 : Quel est le nom de l'application vulnérable ?

PHPMailer

Q2 : A quoi sert (ou comment est utilisée) cette application ? (Utiliser un moteur de recherche)

PHPMailer est une bibliothèque logicielle d'envoi d'e-mails en PHP.

Q3 : Quelles sont les versions vulnérables ?

de 6.1.8 à 6.4.0

Q4 : Quelle est la cause de cette vulnérabilité ?

La version 6.1.8 a introduit un bug qui autorise les chemins UNC.

Q5 : Quelle est la catégorie de vulnérabilité parmi :

- Authentification
- Autorisation
- Cryptographie
- Corruption de mémoire
- Injection de code
- Gestion de session

Injection de code car injection d'objets permet d'exécuter du code.

Q6 : Comment pourrait-on exploiter cette vulnérabilité ? (le principe général)

Envoyer un email qui contient en pièce jointe un fichier phar et l'utilisation du chemin UNC (tel que \\IP-attaquant\).

Exercice 2

A partir de la description suivante, répondre aux questions :

An issue was discovered in GFI Kerio Control 9.2.5 through 9.4.5. The dest GET parameter passed to the /nonauth/addCertException.cs and /nonauth/guestConfirm.cs and /nonauth/expiration.cs pages is not properly sanitized before being used to generate a Location HTTP header in a 302 HTTP response. This can be exploited to perform Open Redirect or HTTP Response Splitting attacks, which in turn lead to Reflected Cross-Site Scripting (XSS). Remote command execution can be achieved by leveraging the upgrade feature in the admin interface.

Q1 : Quel est le nom de l'application vulnérable ?

GFI Kerio Control

Q2 : Retrouver une page web qui explique comment exploiter cette vulnérabilité.

<https://karmainsecurity.com/hacking-kerio-control-via-cve-2024-52875>

Q3 : Quelle extension de fichier doit être utilisée pour exécuter du code sur le serveur ?

.img

Q4 : Quels sont les droits obtenus de l'utilisateur en cas d'exécution de code réussie ?

root (Administrateur)

Exercice 3

Retrouver la vulnérabilité suivante sur le site NVD du NIST : CVE-2021-44228

Q1 : Quel est le nom de l'application vulnérable ?

Apache Log4j

Q2 : Quelles sont les versions vulnérables ?

Version 2.0-beta9 à 2.15.0 (excluding security releases 2.12.2, 2.12.3, and 2.3.1)

Q3 : Quel est son score CVSS v3 ?

10.0 (CRITICAL)

Q4 : Pourquoi cette valeur est-elle si élevée ?

Le nombre d'applications utilisant Log4j est très élevé. L'impact concerne la disponibilité, l'intégrité et la confidentialité. Elle ne nécessite pas d'authentification.

Q5 : Quelle est sa date de publication ?

10 Décembre 2021

Q6 : Quelle est la date et l'heure de la dernière modification de cette CVE ?

06 Novembre 2023

Exercice 3

Retrouver la vulnérabilité suivante sur le site NVD du NIST : CVE-2021-4259

Q1 : Quel est le nom de l'application vulnérable ?

phpRedisAdmin

Q2 : Quelles sont les versions vulnérables ?

Version jusqu'à 1.16.1

Q3 : Quel est le score CVSS v3 donné par NVD ?

9.8

Q4 : Quel est le nom du fichier vulnérable ?

includes/login.inc.php

Q5 : Quel est le numéro de ligne dans ce fichier qui est vulnérable ?

Ligne 59

Q6 : Quelle est la correction faite dans le patch ? (le *commit*)

Ajouter le caractère "=" à "!=" dans un "if".

Q7 : En regardant le code du *commit*, quelle serait la technique générale pour exploiter cette vulnérabilité ? (sans rentrer dans les détails techniques)

En comparant 2 types différents, on pourrait rentrer dans le if. Vérifier sur <https://www.php.net/manual/fr/types.comparisons.php>

Q8 : La plupart du temps, cette erreur commune a peu de conséquences. Pourquoi est-elle plus élevée dans ce cas-là ? Si vous avez trouvé, donner le nom de la catégorie de vulnérabilité.

Ici, on peut potentiellement contourner l'authentification. Catégorie : authentification.

Exercice 4

Effectuer une recherche sur le site NVD du NIST pour afficher les CVE de : Tomcat version 9.0.31

Q1 : Combien de CVE ont été trouvées ?

24 (au 05/02/2024)

Q2 : Quelle est la CVE qui a le CVSS v3.1 le plus élevé ?

CVE-2020-8022

Q3 : Que permet de faire cette CVE pour l'attaquant ?

Elever ses privilèges vers root.

Q4 : Quelle est la condition (pour l'attaquant) pour exploiter cette CVE ?

Etre déjà connecté au système car c'est une élévation de privilèges.

Q5 : En regardant le code d'exploitation, quelle est la difficulté d'exploitation de cette CVE ?

Faible, il suffit d'écrire dans un fichier puis de redémarrer la machine.

Exercice 5

Aller sur : <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator>

Q1 : Déterminer le CVSS v3.1 en fonction des métriques suivantes :

AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

Base score : **7.5**

Q2 : Déterminer le CVSS v3.1 en fonction des métriques suivantes :

AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N

Base score : **4.3**

Exercice 6

Aller sur <https://www.exploit-db.com/>

Q1 : Effectuer une recherche avancée pour trouver le type = webapps et tag = LFI/RFI et afficher le résultat correspondant au 22 janvier 2019. Quel est le nom de l'application vulnérable ?

Joomla

Q2 : Combien de paramètres GET faut-il utiliser pour l'exploit ? Quel est le nom du paramètre GET à utiliser pour afficher un fichier du système ?

3 paramètres GET (option, task et file). Il faut injecter dans le paramètre "file".

Q3 : La page donne 2 exemples de fichiers, lesquels ?

../..//configuration.php et ../..//../..//../..//etc/passwd

Il suffit de décoder en base 64 les 2 paramètres « file ».